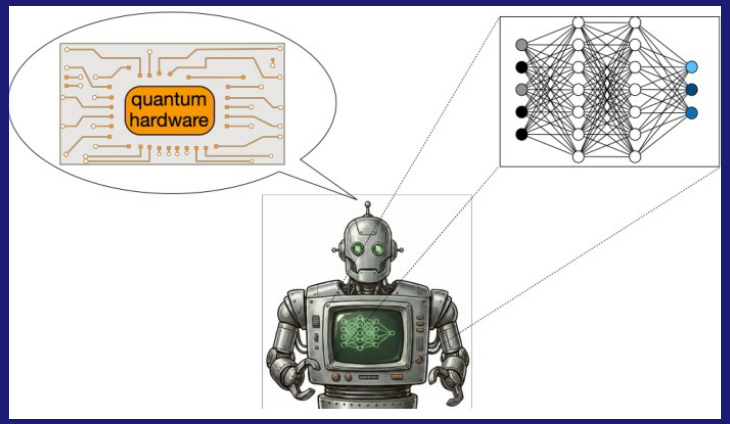
RL in Quantum Applications

From control pulses to feedback, error correction, and quantum agents



Graduate lecture based on Bukov and Marquardt (2026), arXiv:2601.18953

FYS4411 - Reinforcement Learning and Agentic AI

What this lecture is about

A guided map of where RL is actually useful in quantum technology.

- Translate quantum protocols into states, observations, actions, rewards, and episodes.
- Separate three cases: classical RL controlling quantum systems, quantum-enhanced agents, and fully quantum RL.
- Study applications in state preparation, gate design, circuit design, feedback, error correction, and metrology.
- Keep algorithm choices tied to physics: observation model, time scale, reward access, hardware noise, and sample budget.

Why reinforcement learning in quantum technology?

Quantum technology contains many sequential decision problems:

- choose a control pulse over time;
- prepare a target quantum state;
- design a high-fidelity gate;
- adapt to measurement outcomes;
- correct errors before coherence is lost;
- optimize sensing protocols under noise.

The common structure is

$$S_t \longrightarrow A_t \longrightarrow (R_{t+1}, S_{t+1}).$$

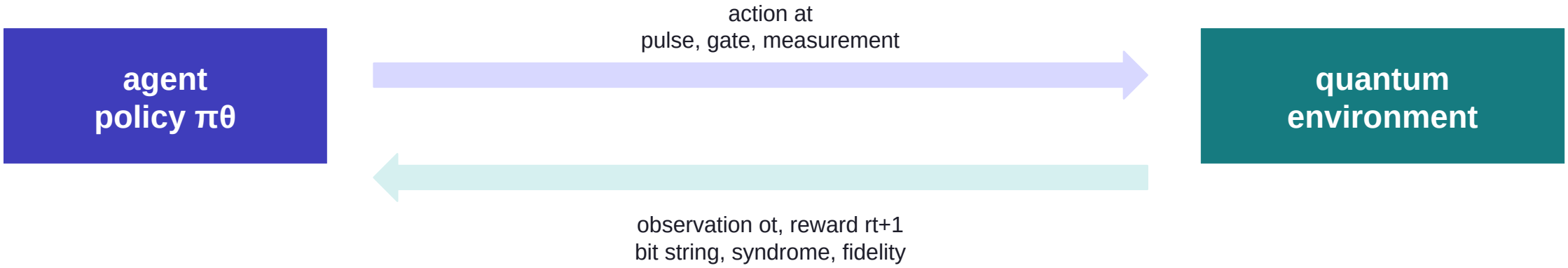
In quantum applications, the "environment" is usually the controlled quantum system, simulator, or experimental device.

FYS4411 — Reinforcement Learning and Agentic AI 1/21

Visual source: attached lecture slide 2; content structure follows Bukov and Marquardt, arXiv:2601.18953.

Why RL enters quantum technology

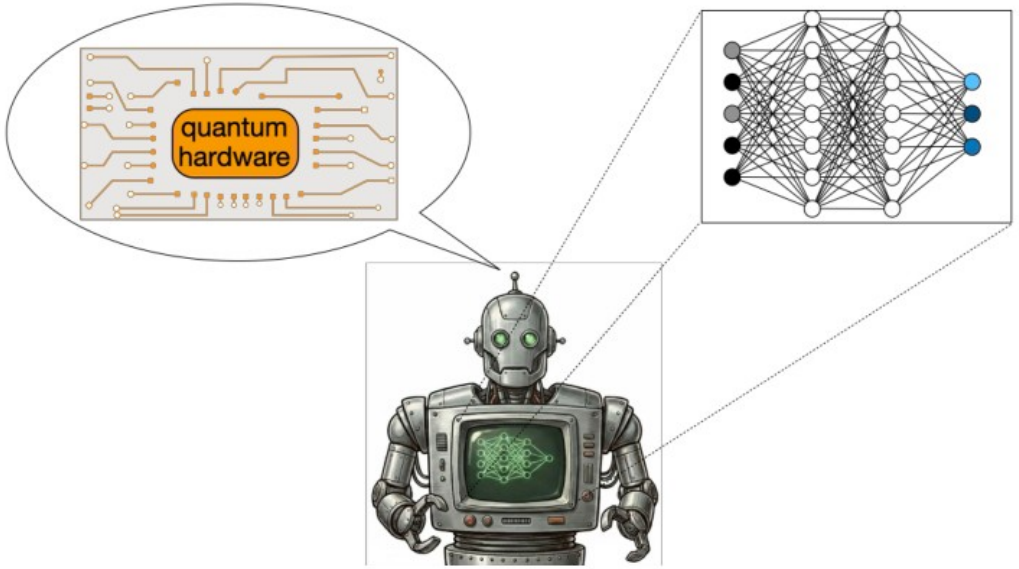
The basic problem is not pattern recognition. It is closed-loop decision making.



Quantum task	Sequential decision
Control	choose amplitudes uk(t) over a finite horizon
Gates	choose pulse pieces or calibration steps
Circuits	append, delete, or route gates
Feedback	adapt after measurement outcomes
QEC	infer and correct errors from syndrome history

The quantum device is the environment

In most applications the agent is classical; the system it controls is quantum.



- The agent never sees the wavefunction in an experiment.
- It sees classical data produced by measurements: bit strings, counts, homodyne records, or syndromes.
- The policy maps available records to the next experimental choice.
- Training may use a simulator, real hardware, or a mixture of both.

$$o_t, \text{history } h_t \rightarrow a_t \rightarrow r_{t+1}, o_{t+1}$$

Figure: Bukov and Marquardt, Fig. 1.

The five application families in the review

They differ mainly in what the action does to the quantum system.

Application	Action	Reward signal	Main bottleneck
State preparation	control Hamiltonian	terminal fidelity	long horizons
Gate design	pulse shape / calibration	gate fidelity	hardware samples
Circuit design	discrete gate edits	energy, compilation cost	combinatorial search
Feedback & QEC	online correction	survival / logical error	partial observability
Metrology	adaptive measurement	information gain / error	Bayesian uncertainty

The paper treats these as one family because each can be written as a finite-horizon decision process interacting with a quantum system.

What is different from textbook RL?

A quantum laboratory is usually a partially observed, stochastic control problem.

- The physical state may be ψt or ρt, but the agent sees observations Ot.
- Measurements are stochastic, and projective measurements change the state.
- Rewards are often terminal: fidelity, energy, lifetime, or logical success after the episode.
- The optimal action may depend on the whole measurement record, not only the last bit.

Quantum RL and the Markov assumption

Quantum RL is Markovian if the state is complete:

$$S_t = \rho_t, \qquad \rho_{t+1} = \mathcal{T}_{a_t}(\rho_t).$$

Then the next step depends only on (S_t, A_t).

In experiments, the agent usually sees observations, not ρ_t:

$$O_t = \langle \sigma_z \rangle_t, \quad m_t \in \{-1, +1\}, \quad \text{syndrome.}$$

So many quantum RL tasks are better explained by a partially observable Markov decision process:

$$A_t \sim \pi(\cdot \mid O_1, \ldots, O_t).$$

FYS4411 — Reinforcement Learning and Agentic AI 8/21

$$at \sim \pi\theta(\cdot \mid o1, o2, ..., ot)$$

Visual source: attached lecture slide 9.

Translation table: RL objects in quantum language

Most design mistakes begin with an ambiguous definition of state and observation.

Quantum RL definitions

RL object	Quantum meaning	Example
State S_t	physical state	ψ_t, ρ_t
Observation O_t	measured information	$\langle\sigma_z\rangle$, syndrome
Action A_t	control operation	pulse, gate, measurement
Reward R_{t+1}	task score	fidelity, energy, survival
Episode	full protocol	one pulse sequence

The lecture goal is to translate quantum problems into this table.

Once this translation is clear, the choice of algorithm becomes a second-order question.

RL object	Quantum interpretation	Example
State St	complete physical state	ρt in a simulator
Observation Ot	measured information	syndrome, expectation value
Action At	control operation	pulse, gate, measurement choice
Reward Rt	task score	fidelity, energy, lifetime
Episode	full protocol	one pulse sequence or QEC cycle

In simulation the state may be accessible; in hardware it normally is not. Design the RL problem from the data stream, not from the mathematical state you wish you had.

Visual source: attached lecture slide 4.

Three environment regimes

The same policy notation hides very different experimental realities.

Simulated
Transitions are generated by numerical time evolution. Rich state information may be available, but model bias is a risk.

Real hardware
The environment is the device. Data are slow, noisy, costly, and affected by drift.

Hybrid
Simulation gives pre-training; the device supplies fine-tuning and calibration.

Three kinds of quantum RL environments

In quantum technology, the RL environment can mean different things:

Type	Data come from	Main issue
Simulated	numerical time evolution	model may be imperfect
Real	quantum hardware	noisy, slow, partially observed
Hybrid	simulation + device data	simulation-to-device transfer

In all cases, the agent only interacts through the RL loop

$$O_t \longrightarrow A_t \longrightarrow (R_{t+1}, O_{t+1}).$$

The practical question is: **what information is actually available to the agent?**

Visual source: attached lecture slide 5.

Model-free does not mean model-free physics

It means the update rule does not require differentiating a known transition model.

$$\rho_{t+1} = \mathcal{E}_{a_t}(\rho_t), \qquad |\psi_{t+1}\rangle = U(a_t)|\psi_t\rangle$$

- The simulator or device still obeys physics; the agent simply treats it as an interaction source.
- This can avoid bias from an inaccurate Hamiltonian or noise model.
- It also creates a sample-efficiency problem: every episode may require many shots or long laboratory time.
- Model-based RL can be attractive when gradients, differentiable simulators, or calibrated models are reliable.

Practical rule
Use model-free methods when the model is suspect or the task is closed-loop; use model-based structure when samples are expensive and the model is trustworthy.

Markov state versus measurement record

The formal Markov state is rarely the interface available to the policy.

$$\text{complete state: } S_t = \rho_t, \quad \rho_{t+1} = T_{a_t}(\rho_t)$$

$$\text{experimental observation: } O_t = \text{measurement record, counts, syndrome}$$

Problem class	Information in the policy	Typical network
Known state simulator	ρ_t or engineered features	MLP / CNN / GNN
Continuous measurement	filtered belief or record history	RNN / transformer / filter + policy
QEC	syndrome history and geometry	CNN / GNN / recurrent decoder
Circuit search	partial circuit and hardware graph	tree search / GNN / policy net

Algorithm choice: start from the physics constraints

The paper reviews algorithms, but the correct first question is what data the laboratory can produce.

Constraint	Implication
Discrete actions	Q-learning, DQN, tree search, categorical policies
Continuous pulses	policy gradients, actor-critic, evolutionary strategies
Sparse terminal reward	reward shaping, curriculum, model-based rollouts
Partial observation	belief states, recurrence, filtering, history windows
Few hardware shots	Bayesian optimization, surrogate models, cautious exploration
Need online feedback	low-latency policy, compact network, robust training

The algorithm is a numerical instrument. The physics defines the observation, action bandwidth, noise, and allowed failure modes.

Policy-gradient methods

Directly optimize the policy that chooses experimental actions.

$$J(\theta) = E\pi\theta\ [\ \Sigma t\ \gamma\text{^}t\ rt{+}1\ ]$$

$$\nabla\theta\ J \approx \Sigma t\ \nabla\theta\ \log\ \pi\theta(at|ht) \cdot Gt$$

- Natural for continuous control amplitudes and stochastic policies.
- REINFORCE is simple but noisy; baselines reduce variance.
- PPO and related actor-critic methods stabilize updates by limiting policy changes.
- Works well when the action distribution must remain smooth and constrained.

Quantum-use pattern

~~Prepare a state, run the protocol, estimate a scalar reward, then update the pulse-selection policy.~~

Value-function methods

Learn which action is promising in a given observed situation.

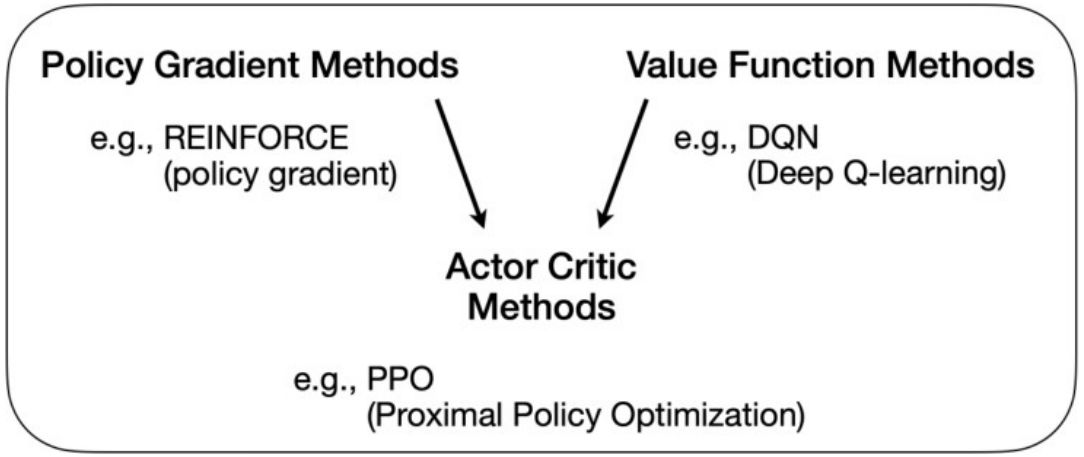
$$Q\pi(h,a) = E\pi\ [\ \Sigma k\ \gamma^k\ rt+k+1\ |\ ht=h,\ at=a\ ]$$

- DQN is attractive when actions are discrete: choose a gate, measurement basis, or correction operation.
- The learned value acts as a ranking of candidate actions.
- Off-policy data can be reused, which can matter when experiments are expensive.
- Instability appears when observations are nonstationary, rewards are sparse, or the action space is very large.

Good match	Less natural
QEC corrections, circuit compilation, discrete control grids	raw continuous microwave pulse synthesis without discretization

Actor-critic methods

A policy is trained with help from an estimated value function.



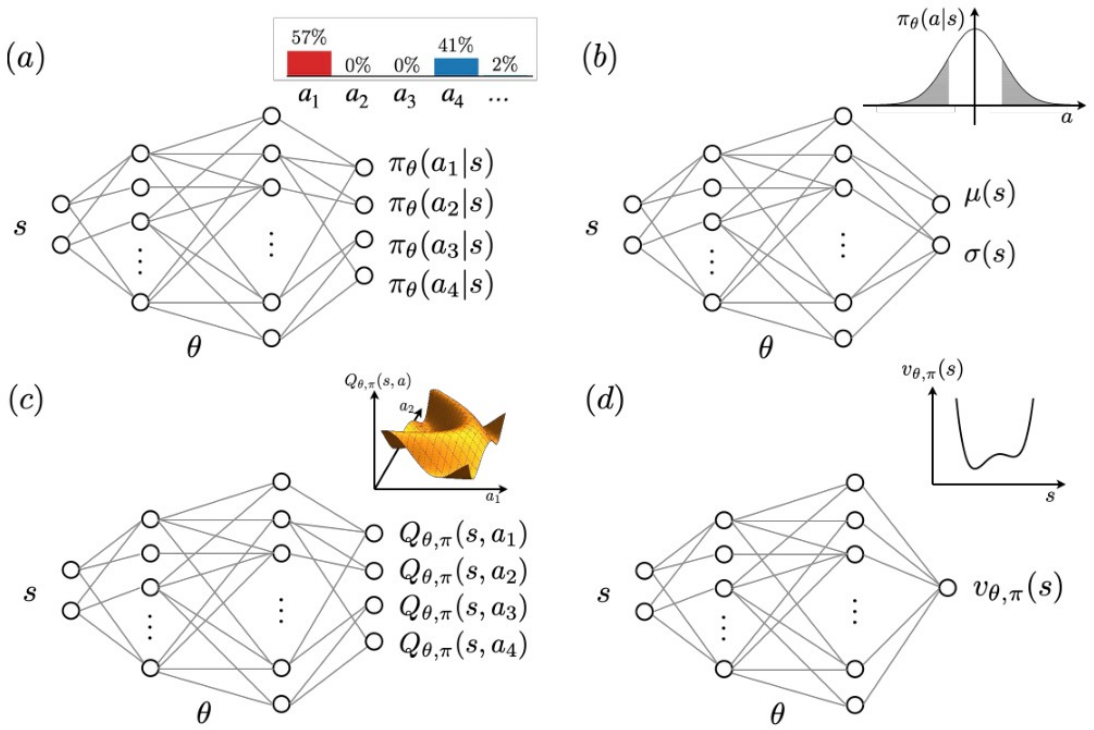
- The actor proposes actions; the critic estimates whether the outcome is better than expected.
- Useful compromise between direct policy optimization and value-based learning.
- PPO is common because it is robust to noisy gradient estimates.
- For quantum tasks, the critic can absorb long-horizon credit assignment that a terminal fidelity alone does not expose.

$$\text{advantage} \quad A(h,a) = Q(h,a) - V(h)$$

Figure: Bukov and Marquardt, Fig. 4.

Deep RL architectures for quantum tasks

The network type should match the structure of the data.



Input structure	Architecture
few scalar observables	MLP
lattice or detector grid	CNN
hardware coupling graph / code Tanner graph	GNN
measurement history	RNN / transformer
candidate circuits	sequence model / tree search

The network is an inductive bias. A surface-code decoder and a two-level pulse problem should not receive the same architecture simply because both are RL.

Figure: Bukov and Marquardt, Fig. 5.

Where the algorithms tend to appear

A quick map before the applications.

Application	Common actions	Common algorithms
State preparation	piecewise controls	policy gradient, PPO, Q-learning on grids
Gate design	pulse samples, calibration knobs	policy gradient, evolutionary strategies, Bayesian optimization
Circuit synthesis	discrete gate edits	DQN, policy gradient, Monte Carlo tree search
Feedback control	measurement-conditioned actions	actor-critic, recurrent policies, filters
QEC decoding	corrections, matching decisions	DQN, GNN policy, multi-agent RL
Metrology	adaptive controls/measurements	Bayesian RL, policy gradient, dynamic programming

The review emphasizes that the useful boundary is not “RL versus optimal control”; it is whether the task is sequential, noisy, partially observed, and interactive.

Reward design in quantum applications

A reward is a compressed physics objective. Compression can hide failure modes.

Reward	Meaning	Hidden risk
Fidelity $\|\langle\psi^*\|\psi_T\rangle\|^2$	target-state preparation	blind to path constraints and robustness
Gate fidelity	unitary is close to target	leakage and drift may be missed
Energy $-\langle H\rangle$	variational ground-state search	barren plateaus / local minima
Survival probability	feedback stabilization	may reward trivial freezing
Logical failure rate	QEC decoding	rare-event statistics are expensive
Information gain	metrology	model assumptions enter the posterior

Quantum state preparation

- Drive a quantum system from an initial state to a target state.
- The reward is often terminal fidelity.
- The core difficulty is long-horizon credit assignment under constrained dynamics.

State preparation as finite-horizon control

This is the cleanest bridge between optimal control and RL.

$$H(t) = H0 + \Sigma k\ uk(t)\ Hk$$

$$|\psi 0\rangle \ \text{--protocol u(t)--> } |\psi T\rangle$$

$$RT = |\langle \psi target \mid \psi T\rangle|^2$$

The agent chooses controls at discrete times. The simulator or device returns a reward after the full protocol, and sometimes intermediate observations.

Learning objective and quantum resources

The objective is still to maximize expected return:

$$J(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} \gamma^t r_{t+1}\right].$$

What may change is the mechanism used to achieve this.

- **superposition of actions**
$$|\psi_A\rangle = \sum_a c_a |a\rangle$$
- **quantum environment query**
$$U_E |a\rangle |0\rangle = |a\rangle |r(a)\rangle$$
- **amplitude amplification** to increase the weight of rewarded actions
- **quantum memory or quantum policy circuits**
$$\pi_\theta(a \mid o) \ \rightsquigarrow\ U_\theta,\ \text{measurement outcomes}$$

FYS4411 — Reinforcement Learning and Agentic AI 15/21

Visual source: attached lecture slide 17.

Few-body state preparation

Small Hilbert spaces make the physics visible but still contain the RL difficulty.

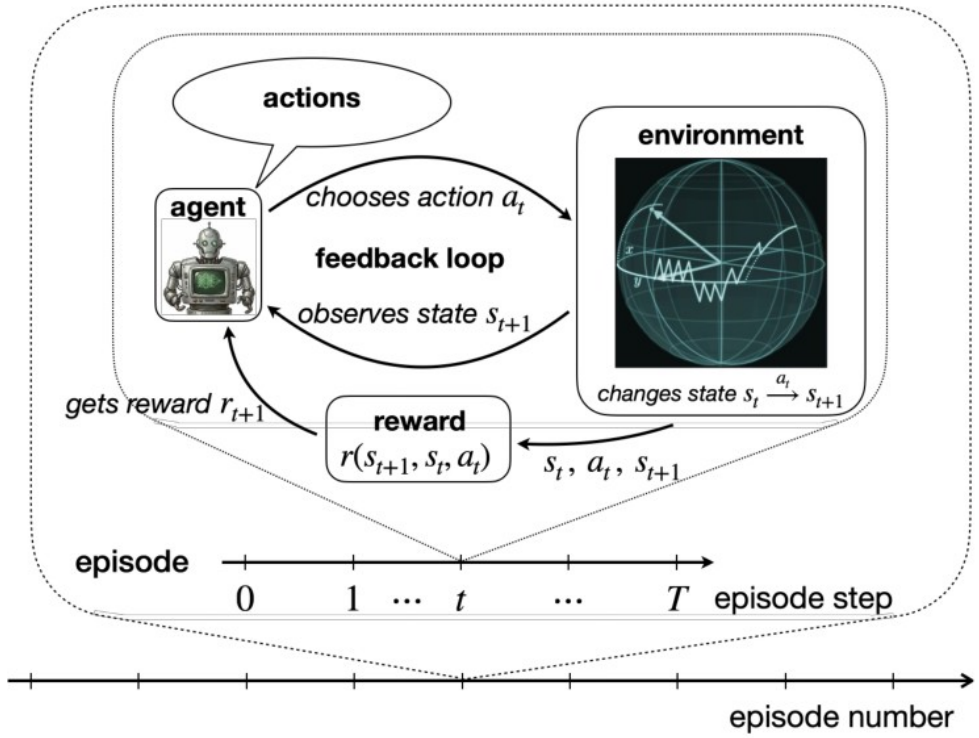
State	Action	Reward
Bloch-vector components, density matrix, or a measurement record.	Discrete rotations, pulse amplitudes, detunings, or measurement bases.	Final fidelity; sometimes shaped by distance to target or leakage penalty.

$$U(aT)...U(a2)U(a1)|\psi 0\rangle \approx |\psi^*\rangle$$

- A Q-learning agent can search a discrete protocol tree.
- A policy-gradient agent can tune continuous amplitudes.
- A model-based planner can use the known Hamiltonian when it is reliable.
- A hardware-trained policy may absorb calibration imperfections that a clean model misses.

Many-body state preparation

The Hilbert space grows exponentially; the policy must exploit structure.



- The action sequence can represent a quench, a ramp, a bang-bang protocol, or a piecewise pulse.
- The terminal reward may be fidelity, overlap with a phase, or an observable proxy when full tomography is impossible.
- Learning can reveal nonadiabatic shortcuts: temporarily leaving a simple path may improve the final overlap.
- Generalization across system sizes requires locality, symmetries, or graph-based representations.

Figure: Bukov and Marquardt, Fig. 3, showing many-body state preparation intuition.

Bang-bang versus continuous protocols

The action space changes the kind of algorithm that is appropriate.

Protocol type	Action representation	Strength	Cost
Bang-bang	choose from a finite set	simple search and value learning	may need many time steps
Piecewise constant	amplitude vector per segment	matches pulse hardware	continuous optimization
Smooth parametrized pulse	few global parameters	low dimension	less expressive
Closed-loop adaptive	condition on measurement history	handles noise online	latency matters

$$a_t \in \{+1, -1\} \quad \text{vs.} \quad a_t = (u1(t), u2(t), \ldots)$$

What RL adds beyond a standard optimizer

RL is useful when the policy itself must act under repeated interaction.

Adaptivity
A trained policy can condition on different initial states, noise realizations, or measurement records.

Exploration
The agent can discover qualitatively different protocols instead of perturbing a known pulse.

Online use
Feedback policies can be deployed in real time after training.

- RL is not automatically superior to GRAPE, Krotov, CRAB, or Bayesian optimization.
- Its advantage appears when the task is partially observed, stochastic, or needs a reusable decision policy.
- For a single deterministic pulse with exact gradients, optimal-control baselines are hard to beat.

Pitfalls in state-preparation RL

A high simulated fidelity can be a poor experimental protocol.

- Reward hacking: the agent exploits the simulator, discretization, or readout proxy.
- Lack of robustness: tiny calibration errors destroy a sharp high-fidelity pulse.
- Tomography burden: estimating fidelity may require many measurements.
- Nonstationarity: drift changes the environment during training.
- Scaling: policies trained on four spins may not control forty without an inductive bias.

Mitigation	How
Domain randomization	train on a distribution of Hamiltonians and noise parameters
Robust reward	penalize leakage, pulse energy, bandwidth, and sensitivity
Curriculum	increase horizon or system size gradually
Hybrid training	pre-train in simulation, fine-tune on hardware

Pulse-shape engineering for quantum gates

- Here the target is not a state, but a unitary operation.
- The agent searches directly in the space of implementable pulses.
- Hardware experiments make this one of the most compelling application classes.

Gate design as RL

The episode is a pulse sequence; the score is how close the resulting channel is to the target gate.

$$\text{target: } U^* \qquad \text{achieved: } U_\theta(T) \qquad \text{reward} \approx \text{gate fidelity}$$

Object	Gate-design meaning
State/observation	calibration data, tomography estimates, previous pulse segment
Action	next amplitude, duration, phase, detuning, or DRAG component
Reward	average gate fidelity, randomized benchmarking score, leakage penalty
Episode	one full waveform implementation and measurement batch

The key reason to use RL on hardware is that the actual device contains leakage, drift, crosstalk, and nonlinearities that are hard to model accurately.

Single-qubit gates

For one qubit, the challenge is not Hilbert-space size; it is precision and robustness.

- The control Hamiltonian typically includes drive amplitude, phase, and detuning.
- Actions can be continuous pulse samples or coefficients in a basis of pulse shapes.
- Rewards can combine gate fidelity, leakage outside the computational subspace, pulse length, and bandwidth.
- RL can be useful when calibration imperfections make analytic pulse design unreliable.

$$H(t) = \tfrac{1}{2}[\, \Delta(t)\sigma z + \Omega x(t)\sigma x + \Omega y(t)\sigma y \,]$$

Experimental constraint
The policy should output pulses that an arbitrary waveform generator can actually synthesize.

Training constraint
The reward estimate is noisy because it comes from repeated shots or benchmarking sequences.

Two-qubit gates: why the problem gets harder

Entangling gates expose leakage, crosstalk, and nonlocal calibration errors.

- Two-qubit gates require simultaneous control of coupled degrees of freedom.
- The pulse must implement the intended entangling interaction while suppressing unwanted transitions.
- The cost function often depends on randomized benchmarking or tomography, not direct access to the unitary.
- A policy can learn device-specific pulse corrections without an exact Hamiltonian model.

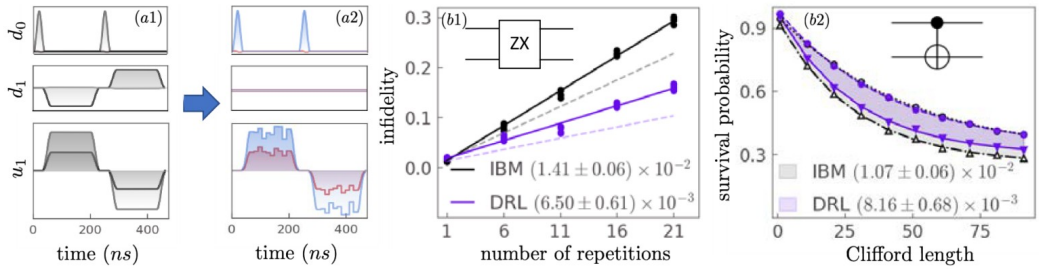
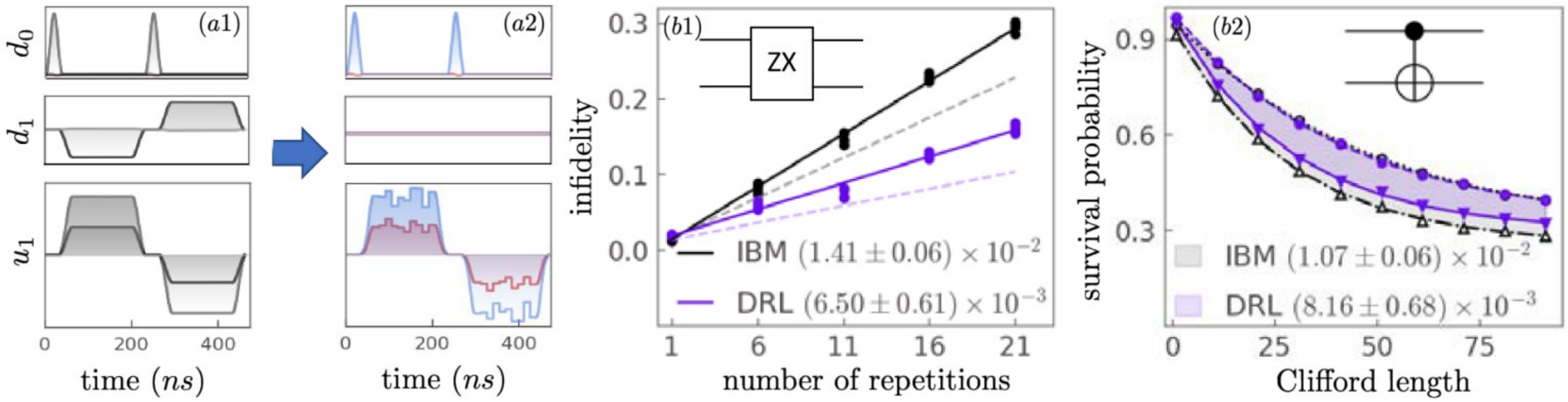


Figure: Bukov and Marquardt, Fig. 8, based on RL-optimized superconducting-gate experiments.

Experimental gate example: cross-resonance control

RL optimized ZX and CNOT gate waveforms on a superconducting quantum computer.



Reported comparison in the figure	Meaning
DRL pulse lower infidelity than state-of-the-art theory for ZX	policy discovers hardware-specific waveform corrections
Robustness tested after training and again after 25 days	checks whether the policy survives calibration drift
Survival probability under Clifford sequences is measured	benchmark focuses on operational gate performance

Figure: Bukov and Marquardt, Fig. 8.

Reward functions for gate design

A single number should reflect the gate you want and the gate you can safely run.

Term	Role in the objective
Average gate fidelity	closeness to target operation
Leakage penalty	discourage population outside computational subspace
Pulse energy / amplitude penalty	avoid hardware saturation and heating
Bandwidth penalty	keep waveforms implementable
Robustness term	average over detuning, drift, or noise distributions
Runtime penalty	favor shorter gates when decoherence is limiting

$$R = F_{gate} - \lambda L_{leakage} - \lambda E_{energy} - \lambda B_{bandwidth}$$

Simulation-to-hardware transfer

A pulse that is optimal in the simulator can fail for precisely the reasons RL is attractive.

Model mismatch

Hamiltonian truncations, neglected levels, calibration drift, crosstalk, and unknown transfer functions.

Safer transfer

Pre-train in a family of randomized models, then fine-tune on the device with cautious exploration.

- Use conservative initial policies on hardware.
- Estimate rewards with uncertainty, not only point values.
- Prefer policies that remain good across a distribution of plausible models.
- Keep physical constraints in the action parametrization rather than hoping the reward will enforce them.

Quantum circuit design

- Actions are discrete symbolic edits to a circuit.
- The search space is combinatorial.
- The physics enters through gate sets, hardware connectivity, and objective functions.

Circuit design as sequential program synthesis

The agent constructs a quantum program one decision at a time.

$$C_0 = I, \qquad C_{t+1} = \text{gate}(a_t) \cdot C_t$$

RL object	Circuit-design meaning
Observation	current circuit, cost, architecture graph, commutation features
Action	append a gate, choose qubits, insert SWAP, terminate, simplify
Reward	energy, fidelity, compilation cost, depth penalty
Episode	construct one candidate circuit or compilation path

This is a natural place for RL because partial circuits have value: a good prefix can make a good final program more likely.

Action spaces for circuits

A small change in the action vocabulary changes the size and bias of the search.

Action vocabulary	What the agent learns
Gate type only	choose from a fixed layer template
Gate + qubit indices	learn both operation and placement
Block insertion	build from reusable motifs
Rewrite rule	simplify or re-route existing circuit
Terminate action	learn the circuit length rather than fixing it

- Small action spaces are easier to train but may exclude good circuits.
- Large action spaces require stronger priors: hardware connectivity, symmetry, locality, or templates.
- Depth penalties matter because today's devices are noise-limited.

$$\text{reward} = \text{accuracy} - \lambda \cdot \text{depth} - \mu \cdot \text{two-qubit count}$$

Entanglement control

RL can learn circuits that create or remove entanglement under hardware constraints.

State-prep view
Build a circuit that creates a target entangled state.

Disentangling view
Find a circuit that maps an entangled state toward a product state.

Compilation view
Replace an expensive circuit by a shallower equivalent circuit.

- The agent may be rewarded for reducing bipartite entanglement entropy or increasing overlap with a simple reference state.
- This connects RL to tensor-network intuition: remove correlations where possible, preserve what is needed.
- Hardware experiments make the problem real because connectivity and gate errors constrain the available moves.

VQE ansatz search

The agent searches over the circuit structure before classical parameter optimization.

Step	What happens
1. Observe	current ansatz, energy estimate, gradients if available
2. Act	add an excitation, entangler, rotation layer, or symmetry-preserving block
3. Optimize parameters	classical inner loop estimates energy
4. Reward	energy decrease minus cost of depth and measurements

$$E(\theta,C) = \langle 0| C^{\dagger}(\theta)\, H\, C(\theta)\, |0\rangle$$

- Circuit structure and continuous parameters form a nested optimization problem.
- The reward should account for measurement cost, not only energy.
- Good action spaces preserve symmetries such as particle number or spin.

Compilation and architecture search

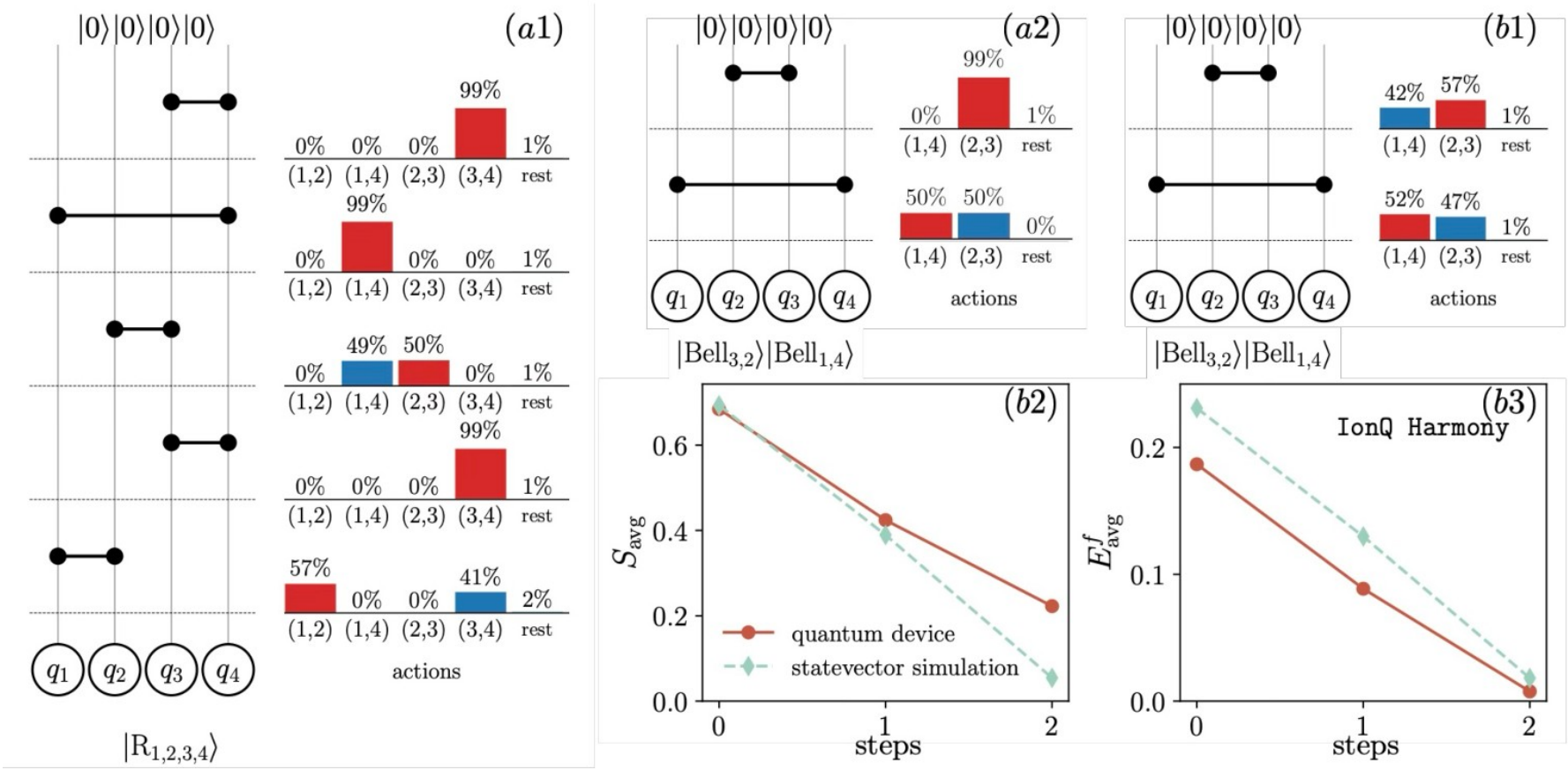
The agent must obey the native gates and connectivity of the processor.

- Transpilation asks for an equivalent circuit with smaller depth, lower error, or feasible routing.
- Architecture search asks for circuit motifs or hardware layouts that perform well across tasks.
- Rewards can combine fidelity estimates, gate errors, qubit connectivity, and run-time constraints.
- The policy can be trained on many random circuits and then deployed on new instances.

Constraint	RL representation
native gate set	mask illegal actions
connectivity graph	graph input or routing action
device noise	weighted cost per gate
circuit identity	terminal equivalence check

Experimental circuit design example

RL-designed disentangling circuits were implemented on a trapped-ion quantum computer.



The figure illustrates a full loop from learned circuit proposals to hardware implementation and measured performance. The important point for RL is that the circuit is not only evaluated symbolically; it is tied to a physical platform.

Figure: Bukov and Marquardt, Fig. 9.

Why circuit RL is hard

The combinatorics are only one part of the problem.

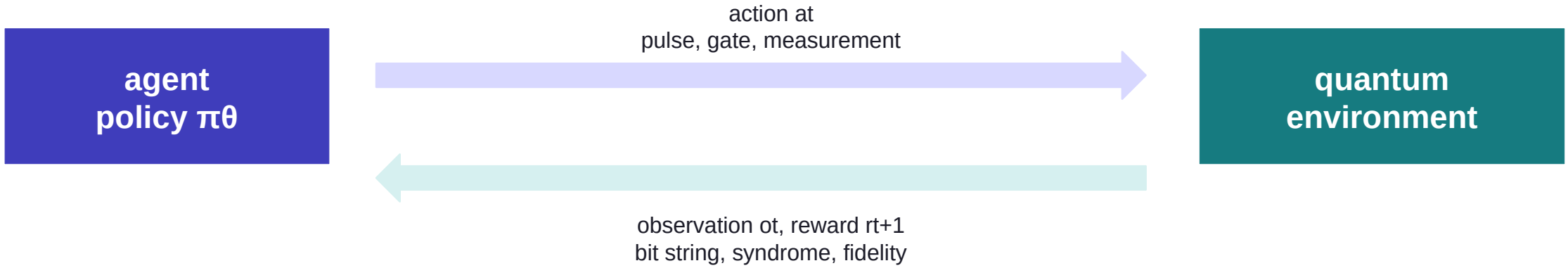
Issue	Why it matters
Huge branching factor	gate type × qubit choice × parameter choice
Delayed reward	bad prefixes can look good and good prefixes can look bad
Equivalence classes	many different circuits implement the same unitary
Noise dependence	a mathematically shorter circuit may be worse on hardware
Measurement cost	estimating energy or fidelity can dominate training time
Interpretability	learned circuits may be hard to simplify or trust

Quantum feedback control

- The agent acts while the quantum system evolves.
- Measurement outcomes are part of the state of knowledge.
- This is where partial observability is not a nuisance, but the problem itself.

Why feedback is a natural RL problem

A feedback controller must respond to a measurement record, not a known trajectory.



Feedback ingredient	RL interpretation
measurement backaction	observation changes the hidden state
continuous monitoring	history or filtered belief is the policy input
actuator bandwidth	action latency and smoothness constraints
long-term objective	reward is survival, stabilization, or event suppression

Quantum trajectories and belief states

The policy often needs a classical estimate of an unobserved quantum state.

$$\text{measurement record: } dy_t = \langle c + c^\dagger \rangle_t \, dt + dW_t$$

$$\text{belief update: } \rho_t \rightarrow \rho_{t+dt} \text{ conditioned on } dy_t$$

- A filter converts noisy measurement records into a belief state.
- The RL policy can act on the belief state, the raw history, or features extracted from the record.
- The separation between “estimation” and “control” is sometimes useful, but learned policies can combine them.
- Feedback experiments impose latency constraints: the policy must run faster than decoherence.

Experimental feedback control

Deep neural networks have been used for real-time feedback control of superconducting qubits.

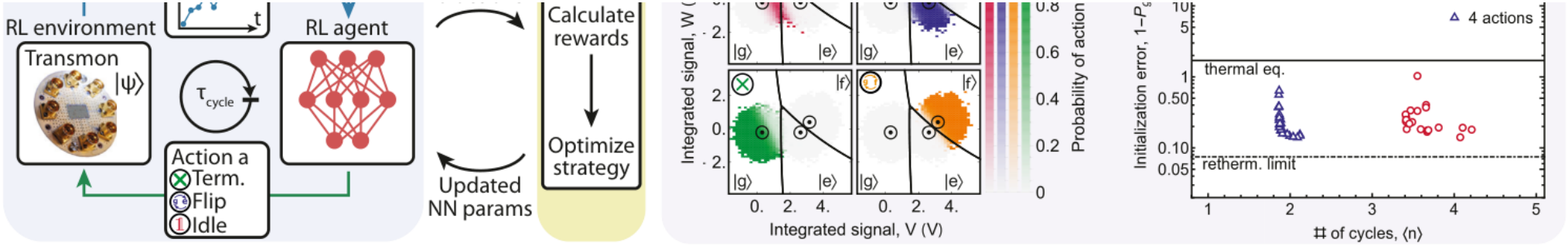


FIG. 10 **Deep neural network agent for real-time feedback control of a superconducting qubit in an experiment.** (a) Outline of the experimental setup and its interaction with the RL agent. In this setup, a response time of less than one microsecond was achieved. (b) Analysis of resulting strategy for ground state preparation in a three-level system (qutrit). The measurement signal is projected onto two variables V and W, and the panels illustrate how the probabilities for various actions (microwave pulses) depend upon the observed measurement result, as suggested by the fully trained agent. (c) Initialization error vs. average number of feedback cycles ("3 actions": the agent is not allowed to flip the transmon directly from state f to state g). Figure adapted with permission from Ref. (Reuer *et al.*, 2023).

particularly in need of the power of RL, since feedback strategies live in an exponentially larger space than mere

times are on the order of nanoseconds, and even a single measurement can be completed during a few hundred

The lesson is architectural as much as algorithmic: a feedback policy must interface with measurement electronics, limited observation bandwidth, and the device time scale.

Figure: Bukov and Marquardt, Fig. 10.

Measurement backaction changes the control problem

In feedback, observing is itself a physical action.

Classical control intuition	Quantum feedback complication
measure state, then act	measurement changes the state
noise is external	noise may come from measurement shot noise and backaction
full state may be estimated	unobserved coherences and leakage remain hidden
reward may be dense	success often measured only at the end

$$\text{policy input} = \text{history } h_t = (o_1, a_1, \ldots, o_t)$$

A recurrent network is not a fashionable add-on here; it is a way to represent the hidden state inferred from a noisy record.

Reward shaping for feedback

Dense rewards can stabilize training, but must not change the physics objective.

Objective	Reward example	Potential trap
keep a qubit in $\|1\rangle$	population at each time step	policy may suppress informative measurements
stabilize an oscillator state	distance to target manifold	penalizes useful transient excursions
protect a code space	syndrome-free intervals	ignores logical errors
detect a jump quickly	early correct detection	false positives can be underweighted

- Always evaluate the final physical metric separately from the training reward.
- Use ablation against filters, threshold controllers, and optimal-control baselines.
- Test robustness under drift and changed noise statistics.

Quantum error correction

- QEC is an online inference-and-control problem.
- The agent observes syndromes, not the error itself.
- The action must prevent logical failure before the code loses information.

QEC as a partially observed Markov decision process

The hidden state is the accumulated physical error; the observation is the syndrome.

$$\text{hidden: } E_t \qquad \text{observed: } s_t = H \cdot e_t \qquad \text{action: correction } c_t$$

RL object	QEC meaning
State	true data-qubit error and measurement-error history
Observation	syndrome bits over one or more rounds
Action	Pauli correction, recovery class, or decoder decision
Reward	no logical failure, reduced residual error, longer memory time
Episode	many QEC cycles under a stochastic noise process

Decoder design choices

The policy can decode directly or learn a component inside a larger decoder.

Design	Action	Strength	Risk
Direct correction	choose Pauli corrections	end-to-end learning	large action space
Syndrome clearing	local moves reducing syndrome	local credit assignment	may create logical operators
Assist MWPM	edge weights or priors	uses proven decoder structure	less autonomous
Multi-agent	local check agents coordinate	scales with geometry	coordination problem
GNN decoder	message passing on Tanner graph	uses code structure	latency and generalization

For fault-tolerant operation, inference quality is not enough. The policy must be fast, stable under changing noise, and implementable in the controller stack.

Real-time constraints for neural decoders

A QEC decoder is useful only if it is faster than the correction deadline.

Latency
Inference must finish within the QEC cycle time or before feed-forward decisions are needed.

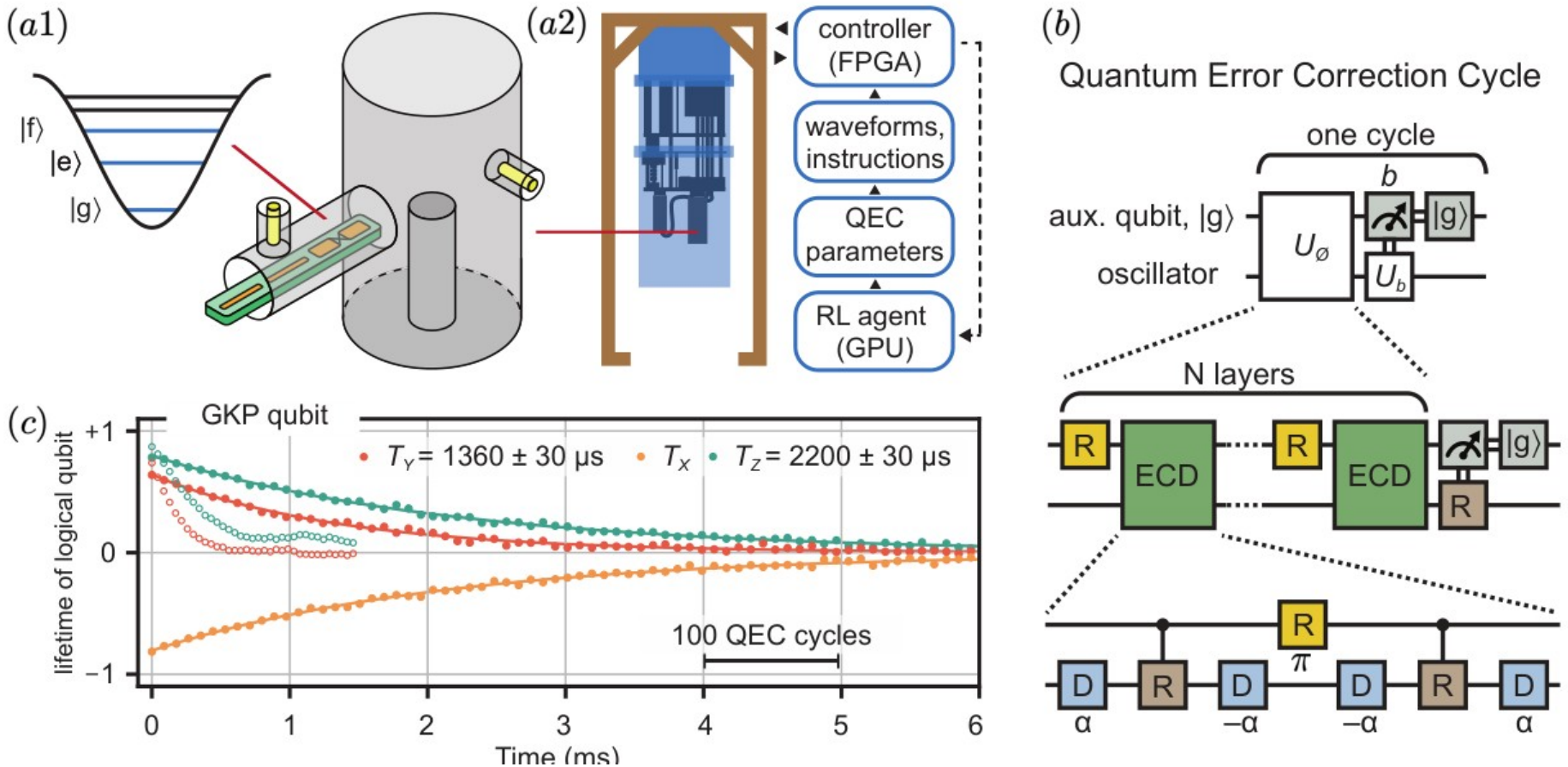
Throughput
The decoder must process many syndrome bits every round as the code grows.

Reliability
Rare logical failures matter; evaluation requires many long trajectories.

- Sparse, local architectures are attractive because QEC checks are local or graph-structured.
- Training can use simulated syndromes, but deployment must tolerate hardware-specific correlations.
- Interpretable failure analysis matters: a decoder that fails silently is dangerous.

Experimental QEC and learned decoding

RL and neural policies are being connected to concrete error-correction cycles.



In QEC, the most relevant observations are not isolated syndromes but syndrome histories with measurement errors. This naturally pushes the problem toward recurrent or graph-structured policies.

Figure: Bukov and Marquardt, Fig. 11.

Discovery of error-correcting codes

RL can also search over the code itself, not only decode a fixed code.

Search object	Example action
Stabilizer generators	add, remove, or modify a check
Connectivity	place checks on a hardware graph
Gauge choices	choose measurement schedule or gauge fixing
Code family parameters	optimize distance, rate, and check weight

- Reward can combine logical error rate, number of physical qubits, check weight, and measurement depth.
- The search space is discrete and constrained by commutation: checks must be mutually compatible.
- A generated code still needs theoretical analysis: distance, thresholds, decodability, and hardware layout.

Graph policies for modern QEC codes

The Tanner graph gives the policy the right inductive bias.

$$\text{CSS code: } H_X z = \text{syndrome}_X, \qquad H_Z x = \text{syndrome}_Z$$

- Variable nodes are qubits; check nodes are stabilizer measurements.
- Message passing can represent local syndrome correlations and code geometry.
- For LDPC and bivariate-bicycle codes, a graph policy is more natural than a dense vector policy.
- A multi-agent decoder can assign local decisions to checks or qubits, then coordinate through graph messages.

Design target
~~Learn a policy that generalizes across rounds, noise strengths, and possibly code sizes without rebuilding the full~~ network.

Evaluation in QEC is rare-event statistics

Good performance means a low logical failure rate, not a low training loss.

Metric	What it tests
Residual error weight	whether local corrections reduce physical errors
Syndrome clearing	whether stabilizer violations disappear
Logical failure rate	whether the encoded information survives
Threshold / pseudo-threshold	scaling with code distance and physical error rate
Latency	whether the decoder can run in real time

- A policy can clear the syndrome and still apply a logical operator.
- Logical failures are rare in the useful regime; estimating them needs many samples.
- Evaluation must include measurement errors, correlated noise, and drift if the hardware has them.

success = no logical error after T rounds

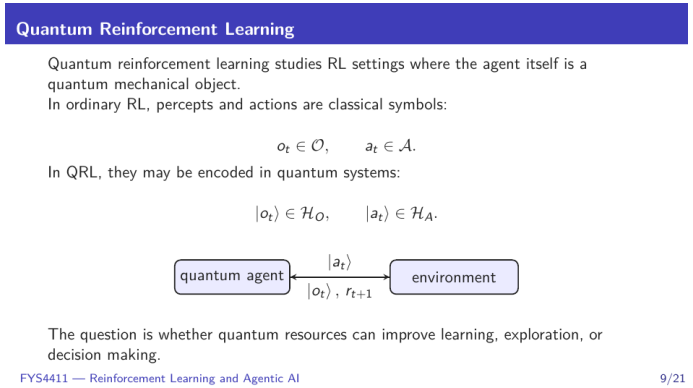
When the agent itself uses quantum resources

- Do not confuse two cases: RL controlling a quantum system and quantum resources inside the learner.
- The review treats QRL separately from the mainstream quantum-technology applications.
- The clean example is amplitude amplification over actions.

Classical RL on quantum systems versus QRL

Most applications in the lecture so far are classical-agent / quantum-environment problems.

Case	Agent	Environment	Typical example
Classical RL for quantum control	classical neural net	quantum device or simulator	pulse design, QEC decoding
Quantum-enhanced RL	quantum circuit or quantum memory	classical or quantum task	superposition over actions
Fully quantum RL	quantum agent	quantum environment	coherent interaction and quantum communication



Visual source: attached lecture slide 10.

A simple classification

C means classical; Q means quantum. The common applications live in the (C,Q) box.

A simple classification

Let the agent type and environment type be

$$A \in \{C, Q\}, \qquad E \in \{C, Q\},$$

where C means classical and Q means quantum.

Agent	Environment	Interpretation
C	C	ordinary reinforcement learning
C	Q	classical RL applied to a quantum task
Q	C	quantum-enhanced reinforcement learning
Q	Q	fully quantum reinforcement learning

(C, Q) is the usual setting in quantum applications,

(Q, C) and (Q, Q) belong more naturally to QRL, i.e. a quantum agent.

FYS4411 — Reinforcement Learning and Agentic AI 10/21

Agent	Environment	Interpretation
C	C	ordinary RL
C	Q	classical RL applied to a quantum task
Q	C	quantum-enhanced RL
Q	Q	fully quantum RL

This distinction avoids a common confusion: controlling a quantum system with PPO is not automatically “quantum reinforcement learning.”

Visual source: attached lecture slide 11.

Quantum episodes: action and reward registers

The action is placed in superposition before querying the environment.

Classical episodes

Consider a finite action set with losing and winning actions,

$$\mathcal{A} = \{\ell, w\}, \qquad r(\ell) = 0, \qquad r(w) = 1.$$

In a classical episode, the agent prepares one action state,

$$|a\rangle_A |0\rangle_R ,$$

and sends it to the environment.

The environment applies a reward-marking unitary,

$$U_E |a\rangle_A |0\rangle_R = |a\rangle_A |r(a)\rangle_R .$$

The returned state is measured, producing a classical reward.

$$r \in \{0, 1\}.$$

FYS4411 — Reinforcement Learning and Agentic AI 12/21

$$H = HA \otimes HR$$

$$UE\ |a\rangle A\ |0\rangle R = |a\rangle A\ |r(a)\rangle R$$

- A classical episode samples one action and receives one reward.
- A coherent episode can query rewarded and unrewarded actions in superposition.
- The reward qubit can phase-mark winning actions, making amplitude amplification possible.

Visual source: attached lecture slides 13-15.

Amplitude amplification in the RL setting

The idea is Grover-like: increase probability weight on rewarded actions before measuring.

Quantum episodes

In a quantum episode, the action register is prepared in superposition,

$$|\psi\rangle_A = \cos\xi\ |\ell\rangle_A + \sin\xi\ |w\rangle_A .$$

The reward register is prepared as

$$|-\rangle_R = \frac{|0\rangle_R - |1\rangle_R}{\sqrt{2}}.$$

The same environment unitary now phase-marks the winning action,

$$U_E\,|\psi\rangle_A\,|-\rangle_R = (\cos\xi\ |\ell\rangle_A - \sin\xi\ |w\rangle_A)\,|-\rangle_R .$$

A reflection step,

$$U_{\mathrm{Ref}} = 2\,|\psi\rangle\,\langle\psi|_A - \mathbb{I}_A,$$

amplifies the winning action before measurement.

FYS4411 — Reinforcement Learning and Agentic AI 13/21

$$|\psi\rangle A = \cos\xi\ |\ell\rangle A + \sin\xi\ |w\rangle A$$

$$\mathrm{UE}\ |\psi\rangle A\ |-\rangle R = (\cos\xi\ |\ell\rangle A - \sin\xi\ |w\rangle A)|-\rangle R$$

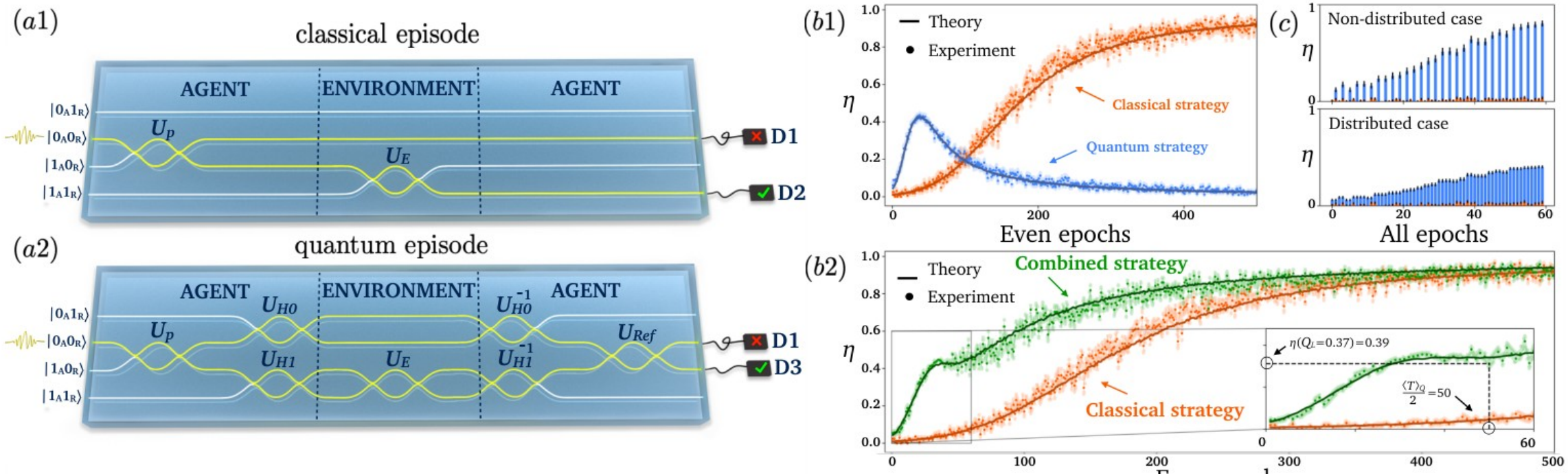
$$\mathrm{URef} = 2|\psi\rangle\langle\psi| - I$$

This does not replace learning. It changes the sampling dynamics of actions, so the agent can find rewarding actions faster in the early stage.

Visual source: attached lecture slide 14.

Nanophotonic quantum-enhanced agent

The paper uses photon exchange as a concrete QRL example.



The key visual comparison is classical episodes versus quantum episodes. The quantum strategy increases reward faster early on, while a combined strategy uses quantum episodes first and classical learning later.

Figure: Bukov and Marquardt, Fig. 12.

Why combine quantum and classical strategies?

Pure amplitude amplification overshoots; classical updating remains useful.

Learning curves: why combine strategies?

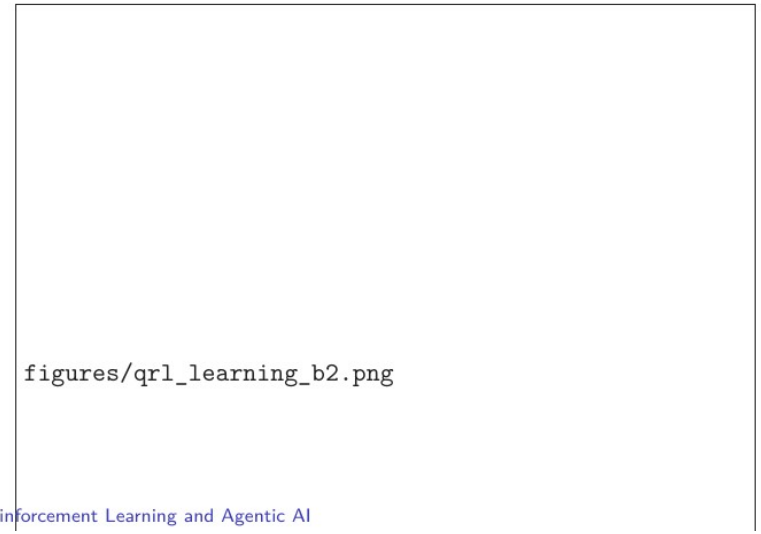
The quantum strategy initially increases the average reward faster.

But it peaks near an optimal episode number j^*, after which performance declines.

This motivates a hybrid strategy.

FYS4411 — Reinforcement Learning and Agentic AI 20/21

Hybrid quantum–classical strategy



FYS4411 — Reinforcement Learning and Agentic AI 21/21

The qualitative lesson is important: quantum resources can help exploration early, but the policy still needs classical reward measurements and updating to settle into a stable high-reward strategy.

Visual source: attached lecture slides 21-22.

Quantum policies and value circuits

Another route to QRL is to represent the function approximator by a variational quantum circuit.

classical policy: $\pi\theta(a|o)$ quantum policy: measure $U\theta(o)|0...0\rangle$

Ingredient	Role
data encoding	map observation to a quantum state or circuit parameters
variational circuit	trainable policy or value approximator
measurement	sample action or estimate value
classical optimizer	update parameters from rewards

- This is attractive when quantum data or quantum subroutines are natural.
- Near-term devices face shot noise, limited qubit counts, and barren plateau issues.
- The benchmark must compare against strong classical networks with the same sample budget.

Limits of QRL claims

The strongest claims require careful accounting of what is queried, measured, and reused.

- A coherent quantum query is not the same resource as one classical laboratory episode.
- Amplitude amplification can speed up search-like subproblems, but it does not solve partial observability or noisy rewards by itself.
- Variational quantum policies can be compact, but compactness is not the same as advantage.
- Any claim of quantum improvement should specify sample complexity, hardware noise, and classical baselines.

Course stance

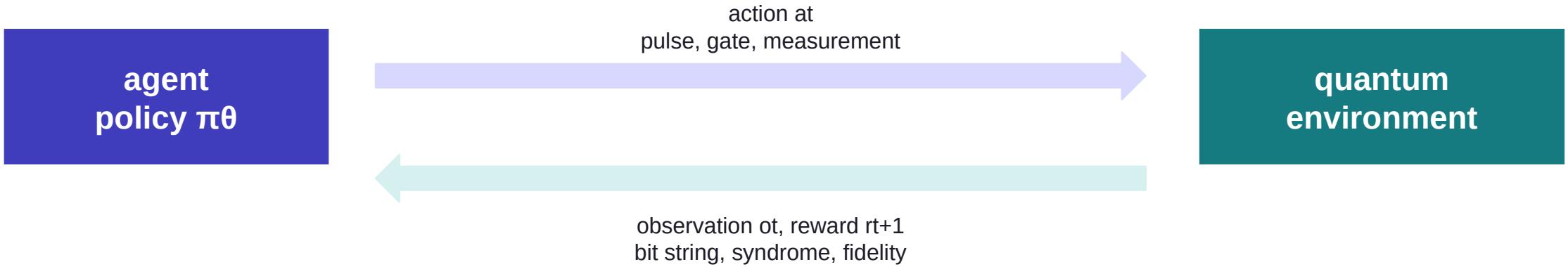
Treat QRL as conceptually important and experimentally interesting, but keep it separate from the immediate success of classical RL for quantum technology.

Quantum metrology and adaptive sensing

- Here the task is to estimate an unknown parameter.
- Actions choose probes, controls, or measurement bases.
- Rewards can be information gain, posterior variance reduction, or final estimation accuracy.

Metrology as adaptive experiment design

An RL policy can choose the next measurement based on what it already knows.



RL object	Metrology meaning
Hidden state	unknown parameter φ or field profile
Observation	measurement outcome conditioned on current probe
Action	probe time, control phase, measurement basis, adaptive feedback
Reward	information gain, reduced posterior variance, low final error

Reward choices for sensing

The reward decides whether the agent behaves like a cautious estimator or an aggressive explorer.

Reward	Policy tendency
negative posterior variance	choose measurements that reduce uncertainty
Fisher information	favor locally informative settings
final squared error	optimize end-to-end estimator performance
classification success	distinguish between hypotheses
cost-sensitive reward	balance precision against time, energy, or decoherence

- Adaptive sensing is often naturally Bayesian: the policy acts on a posterior or sufficient statistics.
- Nonadaptive protocols are strong baselines and should be included.
- The agent must be trained over a prior distribution; changing the prior changes the problem.

Application summary: how to formulate the RL problem

Use this table before selecting an algorithm.

Application	Observation	Action	Reward
State preparation	state features or measurement data	control pulse	final fidelity
Gate design	benchmark/tomography data	pulse segment	gate fidelity - penalties
Circuit design	current circuit + hardware graph	gate or rewrite	accuracy - cost
Feedback control	measurement history or belief	real-time control	survival / stabilization
QEC	syndrome history	correction / decoder decision	no logical failure
Metrology	posterior or outcomes	next probe / measurement	information gain / low error
QRL	quantum/classical percepts	quantum or classical action	task return with quantum resources

Experimental realities

A method that ignores the laboratory bottlenecks is not a quantum-technology method.

Reality	Consequence for RL
shots are expensive	reuse data, use uncertainty, avoid random exploration on hardware
noise drifts	train robustly and monitor policy degradation
measurement is invasive	observation design is part of the physics
latency is finite	feedback policies must be compact and compiled
safety constraints exist	constrain actions before the reward is evaluated
simulators are approximate	evaluate transfer carefully and fine-tune on device

Open challenges emphasized by the review

The hard problems are not solved by naming a stronger network.

- Scalability: Hilbert space, circuit search spaces, and QEC codes grow rapidly.
- Sample efficiency: hardware episodes are expensive and noisy.
- Interpretability: a high-return policy still needs physical understanding and failure analysis.
- Integration: policies must connect to control electronics, compilers, and calibration routines.
- Generalization: policies should survive different initial states, noise regimes, and device instances.
- Benchmarks: compare against strong optimal-control, decoding, compilation, and metrology baselines.

Design checklist for a quantum-RL project

A minimal checklist before training anything.

Question	Answer before training
What is actually observed?	not the wavefunction unless it is a simulator-only study
What actions are physically allowed?	amplitude, bandwidth, latency, gate set, connectivity
How is reward estimated?	shots, tomography, benchmarking, logical trials, posterior updates
What is the baseline?	optimal control, decoder, transpiler, Bayesian estimator, threshold policy
How will transfer be tested?	noise randomization, drift tests, hardware fine-tuning
What failure mode matters?	leakage, logical operator, instability, unsafe pulse, wrong prior

Takeaways and readings

The lecture's main message is formulation before algorithm.

- RL is a language for sequential interaction with quantum systems, not a replacement for physics.
- The most mature applications today are classical agents controlling quantum devices: pulses, circuits, feedback, and decoders.
- Quantum feedback and QEC are naturally partially observed decision problems.
- QRL adds quantum resources inside the agent; it should be distinguished from classical RL applied to quantum hardware.

Primary reading
M. Bukov and F. Marquardt, Reinforcement Learning for Quantum Technology, arXiv:2601.18953 (2026).

RL background
Sutton and Barto, Reinforcement Learning: An Introduction. Focus on MDPs, policy gradients, and value functions.

Quantum control background
Optimal control and open-systems notes for Hamiltonian control, measurement, and noise models.